

textual meaning of sentences, and the representations are sparse. ML-based methods address those issues.

ML-based methods

In these methods, an ML model converts sentences into meaningful word embeddings so that the distance between two embeddings reflects the semantic similarity of the corresponding words. For example, if two words, such as "rich" and "wealth" are semantically similar, their embeddings are close in the embedding space. Figure 4.12 shows a simple visualization of word embeddings in the 2D embedding space. As you can see, similar words are grouped together.

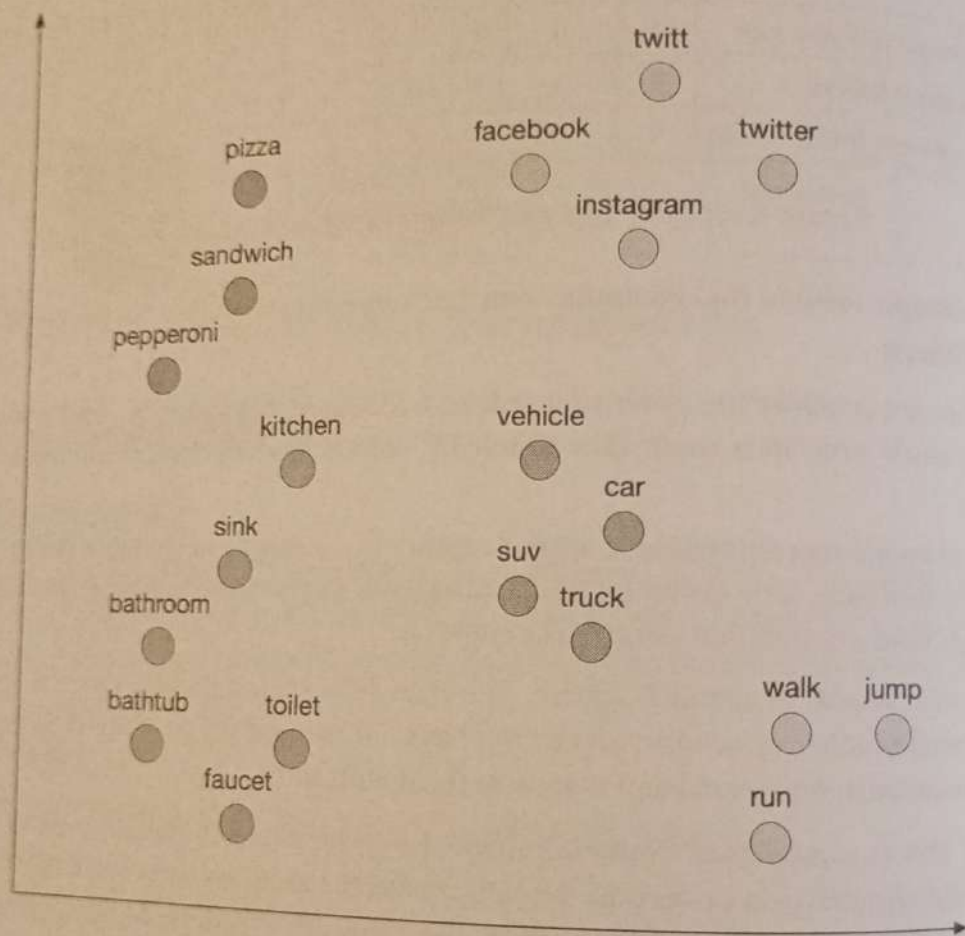


Figure 4.12: Words in the 2D embedding space

There are three common ML-based approaches for transforming texts into embeddings:

- Embedding (lookup) layer
- Word2vec
- Transformer-based architectures

Embedding (lookup) layer

In this approach, an embedding layer is employed to map each ID to an embedding vector. Figure 4.13 shows an example.

4 YouTube Video Search

On video-sharing platforms such as YouTube, the number of videos can quickly grow into the billions. In this chapter, we design a video search system that can efficiently handle this volume of content. As shown in Figure 4.1, the user enters text into the search box, and the system displays the most relevant videos for the given text.

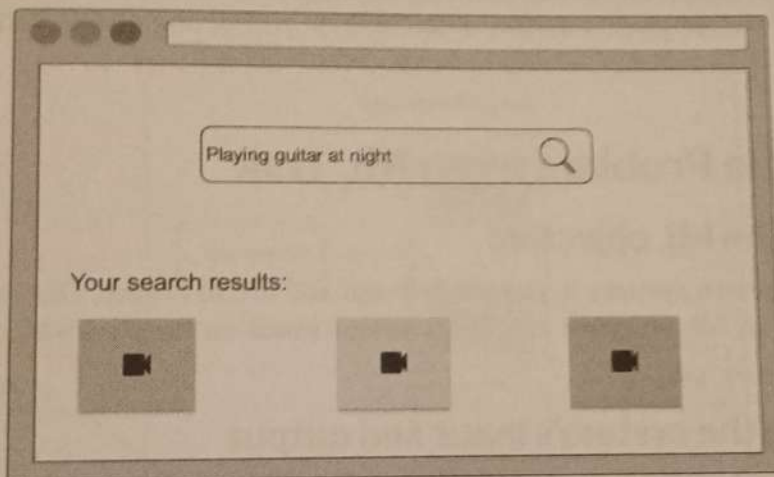


Figure 4.1: Searching videos with a text query

Clarifying Requirements

Here is a typical interaction between a candidate and an interviewer.

Candidate: Is the input query text-only, or can users search with an image or video?

Interviewer: Text queries only.

Candidate: Is the content on the platform only in video form? How about images or audio files?

Interviewer: The platform only serves videos.

Candidate: The YouTube search system is very complex. Can I assume the relevancy of a video is determined solely by its visual content and the textual data associated with the

video, such as the title and description?

Interviewer: Yes, that's a fair assumption.

Candidate: Is there any training data available?

Interviewer: Yes, let's assume we have ten million pairs of (video, text query).

Candidate: Do we need to support other languages in the search system?

Interviewer: For simplicity, let's assume only English is supported.

Candidate: How many videos are available on the platform?

Interviewer: One billion videos.

Candidate: Do we need to personalize the results? Should we rank the results differently for different users, based on their past interactions?

Interviewer: As opposed to recommendation systems where personalization is essential, we do not necessarily have to personalize results in search systems. To simplify the problem, let's assume no personalization is required.

Let's summarize the problem statement. We are asked to design a search system for videos. The input is a text query, and the output is a list of videos that are relevant to the text query. To search for relevant videos, we leverage both the videos' visual content and textual data. We are given a dataset of ten million <video, text query> pairs for model training.

Frame the Problem as an ML Task

Defining the ML objective

Users expect search systems to provide relevant and useful results. One way to translate this into an ML objective is to rank videos based on their relevance to the text query.

Specifying the system's input and output

As shown in Figure 4.2, the search system takes a text query as input and outputs a ranked list of videos sorted by their relevance to the text query.

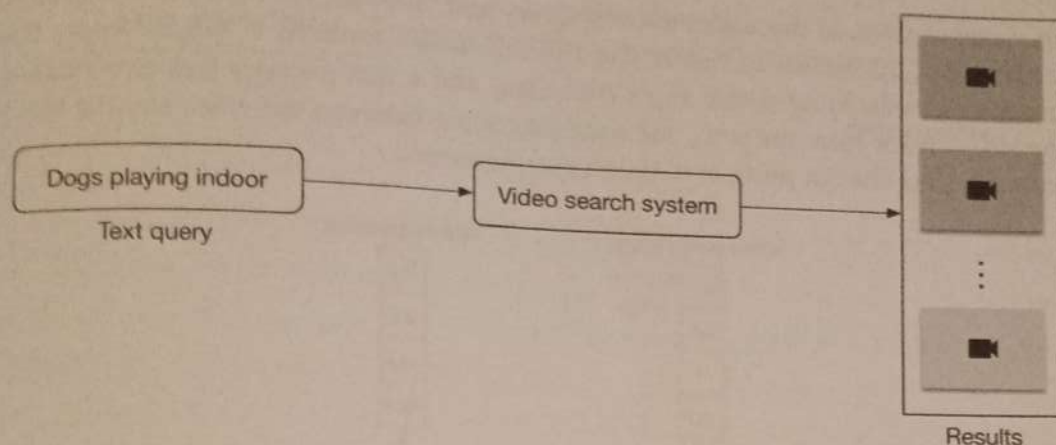


Figure 4.2: Video search system's input-output

Choosing the right ML category

In order to determine the relevance between a video and a text query, we utilize both visual content and the video's textual data. An overview of the design can be seen in Figure 4.3.

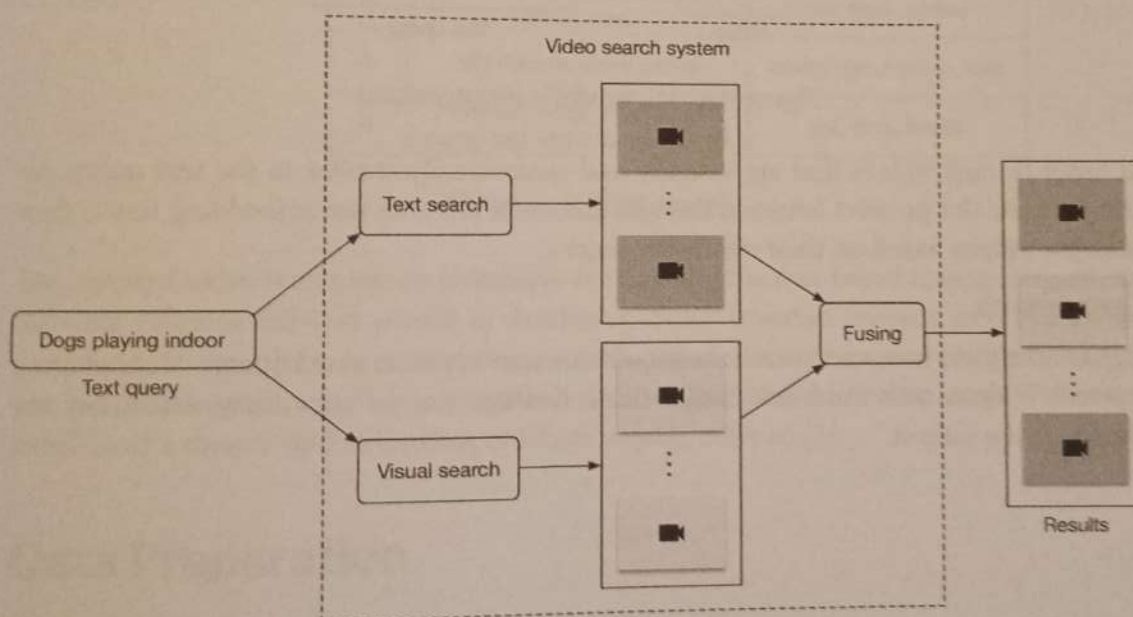


Figure 4.3: High-level overview of the search system

Let's briefly discuss each component.

Visual search

This component takes a text query as input and outputs a list of videos. The videos are ranked based on the similarity between the text query and the videos' visual content.

Representation learning is a commonly used approach to search for videos by processing

their visual content. In this approach, text query and video are encoded separately using two encoders. As shown in Figure 4.4, the ML model contains a video encoder that generates an embedding vector from the video, and a text encoder that generates an embedding vector from the text. The similarity score between the video and the text is calculated using the dot product of their representations.

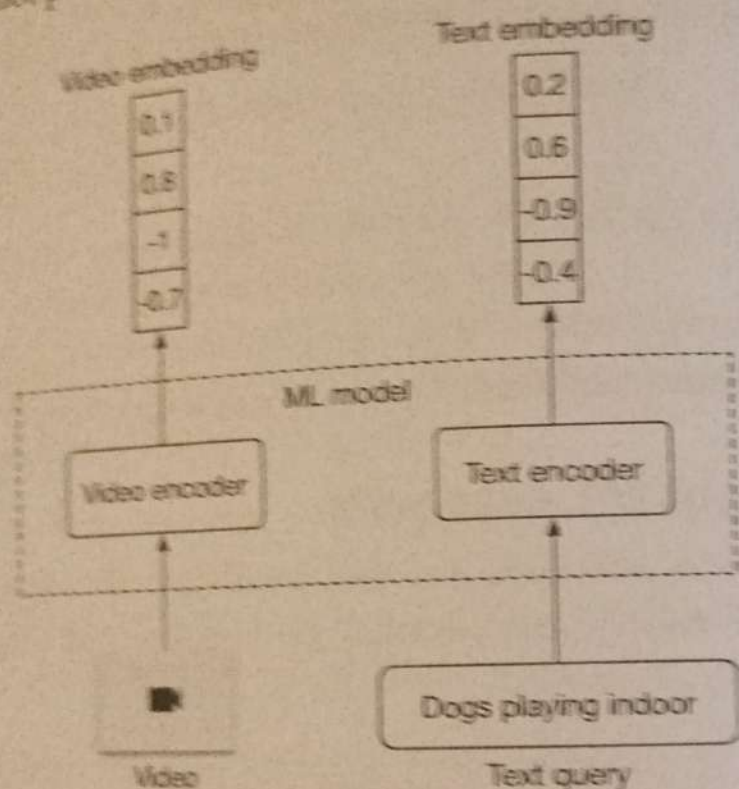
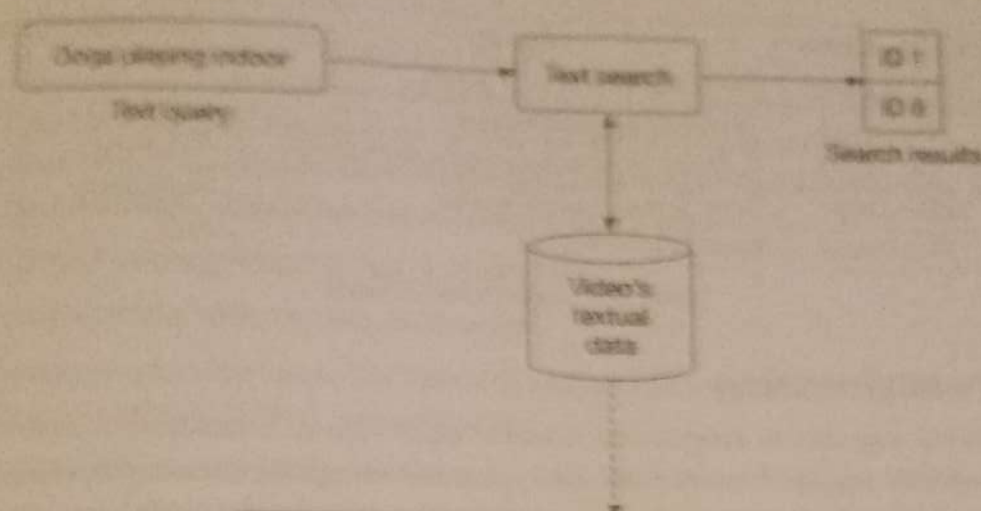


Figure 4.4: ML model's input-output

In order to rank videos that are visually and semantically similar to the text query, we compute the dot product between the text and each video in the embedding space, then rank the videos based on their similarity scores.

Text search

Figure 4.5 shows how text search works when a user types in a text query: "dogs playing indoor". Videos with the most similar titles, descriptions, or tags to the text query are shown as the output.



Video ID	Title	Tags
1	My dog is playing indoor	dog, indoor, play
2	The swimming competition in the middle school	pool, child
3	watch the video I took when I was in Japan	travel, japan, vlog
4	Speed comparison between sport cars	car, race, speed
5	My remote work setup	computer, mouse, talk
6	How our dogs enjoyed the party last night at our house	cat, dog, family

Figure 4.5: Text search

The inverted index is a common technique for creating the text-based search component, allowing efficient full-text search in databases. Since inverted indexes aren't based on machine learning, there is no training cost. A popular search engine companies often use is Elasticsearch, which is a scalable search engine and document store. For more details and a deeper understanding of Elasticsearch, refer to [1].

Data Preparation

Data engineering

Since we are given an annotated dataset to train and evaluate the model, it's not necessary to perform any data engineering. Table 4.1 shows what the annotated dataset might look like.

Video name	Query	Split type
76134.mp4	Kids swimming in a pool!	Training
92167.mp4	Celebrating graduation	Training
2867.mp4	A group of teenagers playing soccer	Validation
28543.mp4	How Tensorboard works	Validation
70310.mp4	Road trip in winter	Test

Table 4.1: Annotated dataset

Feature engineering

Almost all ML algorithms accept only numeric input values. Unstructured data such as texts and videos need to be converted into a numerical representation during this step. Let's take a look at how to prepare the text and video data for the model.

Preparing text data

As shown in Figure 4.6, text is typically represented as a numerical vector using three steps: text normalization, tokenization, and tokens to IDs [2].

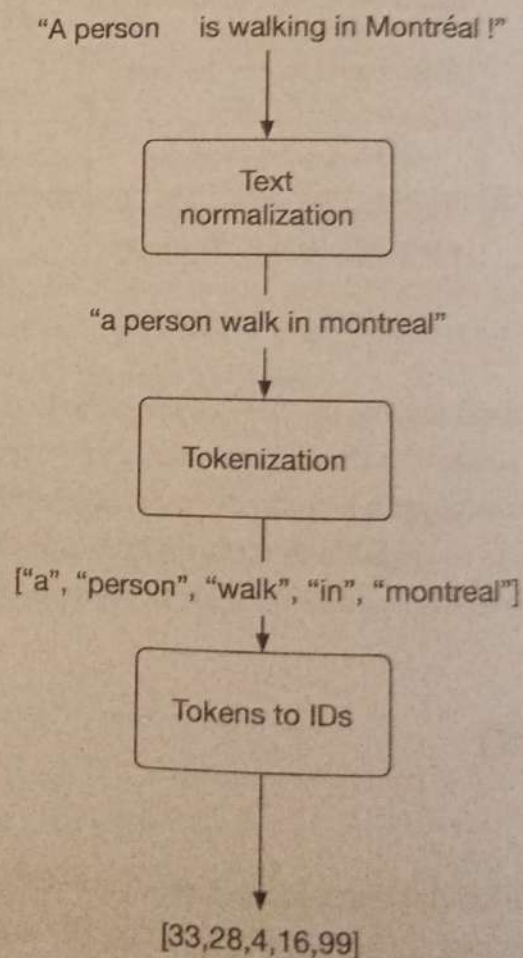


Figure 4.6: Represent a text with a numerical vector

Let's take a look at each step in more detail.

might look like.

Word	ID
⋮	
animals	18
⋮	
art	35
⋮	
car	128
⋮	
insurance	426
⋮	
travel	1239
⋮	

Figure 4.7: A lookup table

Hashing. Hashing, also called “feature hashing” or “hashing trick,” is a memory-efficient method that uses a hash function to obtain IDs, without keeping a lookup table. Figure 4.8 shows how a hash function is used to convert words to IDs.

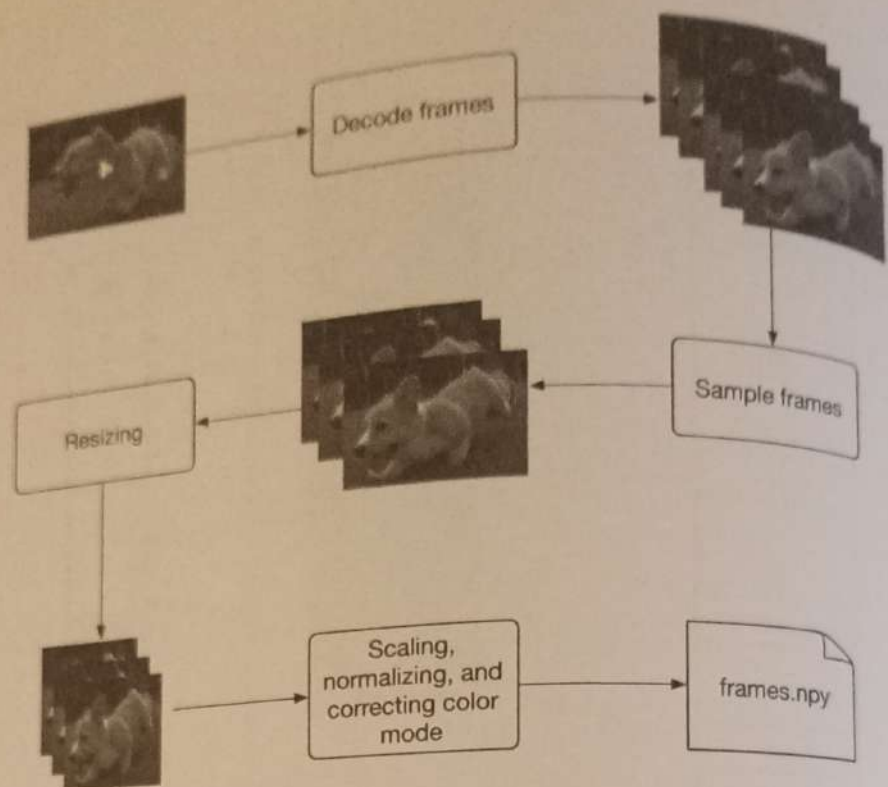


Figure 4.9: Video preprocessing workflow

Model Development

Model selection

As discussed in the “Framing the problem as an ML task” section, text queries are converted into embeddings by a text encoder, and videos are converted into embeddings by a video encoder. In this section, we examine possible model architectures for an encoder.

Text encoder

A typical text encoder’s input and output are shown in Figure 4.10.

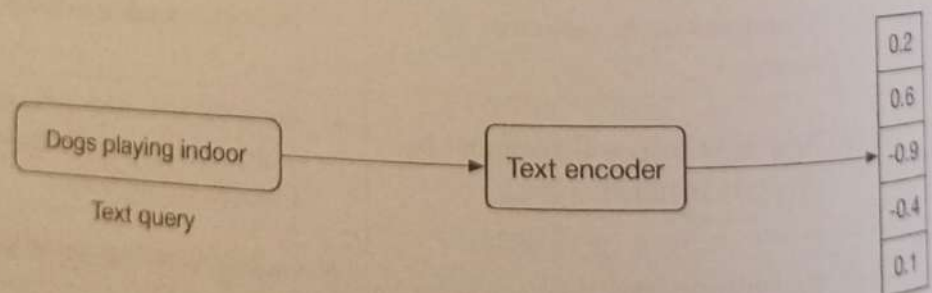


Figure 4.10: Text encoder’s input-output

The text encoder converts text into a vector representation [6]. For example, if two sentences have similar meanings, their embeddings are more similar. To build the text encoder, two broad categories are available: statistical methods and ML-based methods. Let’s examine each.